

**IN1111**

## **Data Structures and Algorithms 1**



### **Smart Courier System**

**Group No : 36**

| <b>Index Number</b> | <b>Name</b>           |
|---------------------|-----------------------|
| 244193M             | Satharasinghe M P     |
| 244178X             | Rathnathilaka M M S P |
| 244039A             | Dahamsa W K M S       |
| 244143L             | Nilupul W A T         |
| 244037P             | Bindusara W A K C     |
| 244114B             | Lakshima T C          |
| 244152M             | Peiris P V S H        |

Bachelor of Science Honors in Information Technology (BSc Hons (IT))

Faculty of Information Technology

University Of Moratuwa

# Table of Contents

|                               |  |
|-------------------------------|--|
| <b>1. Introduction.....</b>   |  |
| <b>2. System Design.....</b>  |  |
| <b>3. Implementation.....</b> |  |
| <b>4. Sample Outputs.....</b> |  |
| <b>5. Discussion.....</b>     |  |
| <b>6. Conclusion.....</b>     |  |
| <b>7. Source Code.....</b>    |  |

# 01. Introduction

## Problem Statement

A Smart Courier Management System is required to efficiently manage parcel registration, tracking, pickup requests, dispatch operations, and delivery routing within a courier service. The system should organize parcel records, maintain detailed tracking histories with timestamps, handle pickup and dispatch processes in proper order, and determine the most efficient delivery route between interconnected centers based on distance. By using appropriate data structures, the system ensures accurate data management, operational efficiency, and automatic updating or removal of parcel records once delivery is successfully completed.

## Objectives

- To develop a Smart Courier Management System using fundamental data structures.
- To manage parcel registration and maintain organized parcel records.
- To maintain detailed tracking history with status updates and timestamps.
- To handle pickup requests using a proper FIFO mechanism.
- To manage parcel dispatch operations efficiently.
- To determine the shortest delivery route between centers based on distance.
- To integrate all system components for smooth workflow.
- To automatically update and remove parcel records once delivery is completed.

## 02. System Design

### 1. Parcels (Doubly Linked List)

This doubly linked list stores information on all active parcel records, of which, each node holds id, sender, receiver, weight, source, destination, and a pointer to its own tracking history. (Another linked list)

The function '*findParcel*' traverses list comparing IDs and returns a pointer to the matching node.

The function '*detachParcelFromList*' deleted a node by connecting prev and next pointers of surrounding nodes without freeing memory of the relevant node. (When the node is not deleted, but merely moved into another data structure)

The function '*deleteParcel*' connects the neighboring nodes together and then frees the node and deletes every reference for the node in other data structures.

### 2. Distribution Centers (Circular Doubly Linked List)

This circular doubly linked list stores all delivery hub locations in order the deliveries take place from center to center. Each node in this list holds center name, distance to next center, distance to previous center.

Function '*addCenter*' inserts a new node after a specified existing node, (Or default, after the tail node) and changes next and prev pointers of 2 nodes and update two distances accordingly.

Function '*calcRoutes*' traverses the circle in both directions from the source node, adding up distances and building a path until the destination is found and then returns both paths and distances.

Function '*findBestRoute*', calls calcRoutes and compares forward and backward distances and print both paths and declares the shorter one as the best route.

Function '*simulateDelivery*' also calls calcRoutes to decide delivery path direction when building the route stack, picks forward if forward distance is less than or equal to backward.

### 3. Tracking History (Singly Linked List, one per Parcel)

Each parcel node contains a pointer to its own independent tracking singly linked list of which, each node holds status, location and timestamp relevant to each tracking state change.

Function '*appendTracking*' is the function used for all tracking records, called automatically by the system at registration, pickup request, dispatch and delivery. (Also, manually by the user via the tracking menu to add location updates mid-delivery)

Function '*freeTracking*' frees each node when a parcel is deleted or when completed stack is cleared.

#### 4. Pickup Queue

A queue that holds parcel IDs of parcels awaiting pickup in request order.

Function '*enqueuePickup*' adds a node to the rear and calls the function '*appendTracking*' with status Pickup Requested and the timestamp entered by the user.

Function '*removeFromQueue*' traverses the queue to find and unlink a specific parcel ID. Used when a parcel is deleted mid-queue.

#### 5. Dispatch Queue

A queue that holds parcel IDs of parcels that have been picked up and are ready for delivery.

Function '*moveToDispatch*' dequeues from "pickup" front and enqueues to dispatch rear. Calls '*appendTracking*' with status "Out for Delivery".

Function '*sortDispatchByWeight*' uses insertion sorting to sort the dispatched parcels by descending order based on weight which helps storing heaviest parcels at bottom to transport them undamaged. It detaches nodes one by one from the original queue and finds the correct position by comparing parcel weights via '*findParcel*', relinks the node there and repeats until all nodes are placed. Then resets front and rear pointers of the queue accordingly.

#### 6. Route Stack

This is a temporary Stack built and destroyed during each delivery simulation. It holds center names in delivery order with LIFO behavior.

Function '*pushRoute*' allocates a new node, sets its center name, and places it on top of Route Stack.

## 7. Completed Deliveries (Stack)

The stack holds pointers to delivered Parcel nodes including their attached tracking histories in case a review of past deliveries is needed. Most recent delivery sits on top, LIFO via top pointer.

Function *'pushCompleted'* receives the detached Parcel pointer directly from *'simulateDelivery'* and places it on top of the stack.

Function *'viewCompleted'* traverses the stack from top and for each node, prints full parcel details followed by every entry in its tracking linked list as well.

Function *'clearCompleted'* traverses the stack popping each Completed Node, freeing the tracking list via *'freeTracking'*. Then frees the Parcel node and then frees the Completed Node itself.

## 03. Implementation

### 1. Parcel Doubly Linked List

#### **addParcel**

Allocates a new Parcel node, reads all fields from user, appends to the tail of the doubly linked list by walking from head until next is NULL, then links the new node. Calls '*appendTracking*' to record the registration entry.

#### **deleteParcel**

Finds the parcel, calls '*removeFromQueue*' on both queues to delete any references, then '*detachParcelFromList*' to unlink from the list, then frees tracking history and the node itself.

#### **findParcel**

Search from head comparing IDs and return pointer to matching node. (or NULL)

#### **detachParcelFromList**

Resets prev and next pointers of surrounding nodes and set the detached node's own pointers to NULL so it can be on the completed stack.

#### **displayParcels**

Traverses doubly linked list from head, printing each parcel's fields.

### 2. Tracking Singly Linked List

#### **appendTracking**

Traverses to tail of the parcel's tracking list and appends a new node and is called automatically at registration, pickup, dispatch and delivery. Can be called manually via tracking menu for location updates.

## **viewTracking**

Find the parcel then traverses its tracking list printing each entry.

## **freeTracking**

Traverses the list freeing every node and is used to delete or clear completed stack.

## 3. Center Circular Doubly Linked List

### **addCenter**

Asks for a center to insert after. (Defaults is after tail)

Resets four pointers: next and prevs of 2 neighboring centers.

Updates 2 distances after removing the previous distance between the center in between which the new center was added to.

### **calcRoutes**

From source node, traverses forward via next adding up distToNext into fwdDist and builds up fwdPath string. Then traverses backward via prev summing up distToPrev into bwdDist and buildings up bwdPath string. Stops each traversal when destination name matches.

### **findBestRoute**

Calls '*calcRoutes*' and prints both paths with distances, prints the one with lower distance as best route.

### **displayCenters**

Do-while loop traversing the circle, stops when back at head.

#### 4. Pickup Queue

##### **enqueuePickup**

Allocate '*QueueNode*' with parcel ID and links to rear, updating rear pointer. Calls '*appendTracking*' with "Pickup Requested" status and user-entered timestamp.

##### **displayPickupQueue**

Traverses from 'front' printing each parcel ID with position number.

##### **removeFromQueue**

Take Queue pointer and parcel ID and then traverses the queue with a prev tracker to unlink the matching node and fix surrounding links.

#### 5. Dispatch Queue

##### **moveToDispatch**

Dequeues from pick up front, enqueues to dispatch rear, calls '*appendTracking*' with "Out for Delivery" status.

##### **sortDispatchByWeight**

*Insertion sorting:* Divides the queue into a sorted portion and unsorted portion within the same list. Starts with the first node considered already sorted. Takes the next unsorted node, retrieves its parcel weight via *findParcel*, then traverses the sorted portion from the front to find the correct position. Relinks the node into that position by adjusting next pointers of surrounding nodes. Repeats until all nodes are in the sorted portion. Updates *dispatchQueue.front* and *dispatchQueue.rear* at the end.

##### **displayDispatchQueue**

Traverses from front, calls *findParcel* for each ID to retrieve and print weight and route alongside the ID.

## 6. Route Stack

### **pushRoute**

Allocates StackNode, copies center name, points its next to current top and sets top to new node.

### **simulateDelivery**

Calls '*calcRoutes*' to compare distances and picks direction. Traverses center list in chosen direction **from destination to source** pushing each center name onto routeStack. (Since it was pushed from destination to source, at the end, source is at the top of the stack) Pops and prints each step. After route printed, asks for delivery timestamp and call '*appendTracking*' with "Delivered" status. Then detaches parcel from the list and pushes to completed stack.

## 7. Completed Stack

### **pushCompleted**

Allocate Completed Node and store the Parcel pointer and then push onto top.

### **viewCompleted**

Traverses stack from top, for each Completed Node, prints full parcel details and traverses that parcel's tracking list printing every entry.

### **clearCompleted**

Pop each Completed Node, then call '*freeTracking*' on the parcel's tracking singly linked list and free the Parcel node and the Complete Node.

## 04. Sample Outputs

Loading Demo Data:

```
MAIN MENU
-----
1. Delivery Centers
2. Parcels
3. Tracking
4. Dispatch & Delivery
5. Completed Deliveries
6. Load Demo Data
0. Exit
-----
Choice: 6
-----
Load Demo Data
-----
Timestamp for all demo entries : 09:00
Centers: Colombo, Gampaha, Anuradhapura, Galle
Parcels: #101 (Colombo->Gampaha, 2.5kg), #102 (Gampaha->Galle, 1.0kg),
         #103 (Galle->Colombo, 4.0kg), #104 (Anuradhapura->Galle, 6.0kg),
         #105 (Colombo->Anuradhapura, 3.2kg)

Demo data loaded.

Press Enter to continue...
```

Display Delivery Centers:

```
MAIN MENU
-----
1. Delivery Centers
2. Parcels
3. Tracking
4. Dispatch & Delivery
5. Completed Deliveries
6. Load Demo Data
0. Exit
-----
Choice: 1
-----
[ Centers ]
-----
1. Add Delivery Center
2. Display Centers
3. Find Best Route
0. Back
-----
Choice: 2
-----
Delivery Centers (Circular)
-----
[1] Colombo --> Gampaha (35.0 km)
[2] Gampaha --> Anuradhapura (180.0 km)
[3] Anuradhapura --> Galle (290.0 km)
[4] Galle --> Colombo (120.0 km)

Press Enter to continue...|
```

Find Best Route:

```
[ Centers ]
-----
1. Add Delivery Center
2. Display Centers
3. Find Best Route
0. Back
-----
Choice: 3
-----
Find Best Route Between Centers
-----
Source Center      : Gampaha
Destination Center : Galle

Forward  : Gampaha -> Anuradhapura -> Galle (470.00 km)
Backward : Gampaha -> Colombo -> Galle (155.00 km)

Best route: Gampaha -> Colombo -> Galle (155.00 km)

Press Enter to continue...|
```

Adding a Center to the middle:

```
-----
Choice: 1
-----
Add Delivery Center
-----
Center Name : Rathnapura
Located after which center (default: Galle, press Enter to use default): Anuradhapura
Distance from Anuradhapura to Rathnapura (km): 200
Distance from Rathnapura to Galle (km): 120
Center 'Rathnapura' inserted after 'Anuradhapura'.

Press Enter to continue...

[ Centers ]
-----
1. Add Delivery Center
2. Display Centers
3. Find Best Route
0. Back
-----
Choice: 2
-----
Delivery Centers (Circular)
-----
[1] Colombo --> Gampaha (35.0 km)
[2] Gampaha --> Anuradhapura (180.0 km)
[3] Anuradhapura --> Rathnapura (200.0 km)
[4] Rathnapura --> Galle (120.0 km)
[5] Galle --> Colombo (120.0 km)

Press Enter to continue...|
```

Display Parcels:

```
[ Parcels ]
-----
1. Register Parcel
2. Delete Parcel
3. Display All Parcels
0. Back
-----
Choice: 3
-----
Parcel Records
-----
ID: 101  Sender: Chathushi  Receiver: Sadeesha
Weight: 2.50 kg    Colombo  -->  Gampaha
-----
ID: 102  Sender: Manula  Receiver: Supun
Weight: 1.00 kg    Gampaha  -->  Galle
-----
ID: 103  Sender: Dahamsa  Receiver: Bindusara
Weight: 4.00 kg    Galle  -->  Colombo
-----
ID: 104  Sender: Thisara  Receiver: Dahamsa
Weight: 6.00 kg    Anuradhapura  -->  Galle
-----
ID: 105  Sender: Amal  Receiver: Kamal
Weight: 3.20 kg    Colombo  -->  Anuradhapura
-----
Press Enter to continue...|
```

Registering a parcel:

```
[ Parcels ]
-----
1. Register Parcel
2. Delete Parcel
3. Display All Parcels
0. Back
-----
Choice: 1
-----
Register New Parcel
-----
Parcel ID    : 106
Sender       : Sunil
Receiver      : Nimal
Weight (kg)  : 2.5
Source       : Anuradhapura
Destination  : Rathnapura
Timestamp    : 10:00

Parcel #106 registered successfully.

Press Enter to continue...|
```

Deleting a parcel:

```
[ Parcels ]
-----
1. Register Parcel
2. Delete Parcel
3. Display All Parcels
0. Back
-----
Choice: 2
-----
Delete Parcel
-----
Parcel ID: 105
Parcel #105 deleted and removed from all queues.
Press Enter to continue...
```

Tracking History:

```
MAIN MENU
-----
1. Delivery Centers
2. Parcels
3. Tracking
4. Dispatch & Delivery
5. Completed Deliveries
6. Load Demo Data
0. Exit
-----
Choice: 3
-----
[ Tracking ]
-----
1. Add Tracking Update
2. View Tracking History
0. Back
-----
Choice: 2
-----
Tracking History
-----
Parcel ID: 101

History for Parcel #101 (Colombo -> Gampaha):

[1] Status: Registered Location: Colombo Time: 09:00
Press Enter to continue...|
```

Adding a tracking Update Manually:

```
-----
Choice: 1
-----
Add Tracking Update
-----
Parcel ID   : 101
Status      : Packaging
Location     : Colombo
Timestamp    : 10:05
Tracking update added for parcel #101.

Press Enter to continue...

[ Tracking ]
-----
1. Add Tracking Update
2. View Tracking History
0. Back
-----
Choice: 2
-----
Tracking History
-----
Parcel ID: 101

History for Parcel #101 (Colombo -> Gampaha):

[1] Status: Registered   Location: Colombo   Time: 09:00
[2] Status: Packaging     Location: Colombo   Time: 10:05

Press Enter to continue...|
```

Pick up requests:

```
[ Dispatch ]
```

- ```
-----  
1. Add Pickup Request  
2. View Pickup Queue  
3. Move to Dispatch Queue  
4. View Dispatch Queue  
5. Sort Dispatch by Weight (Heaviest First)  
6. Deliver Next Parcel (Simulate)  
0. Back  
-----
```

```
Choice: 1  
-----
```

```
Add Pickup Request  
-----
```

```
Parcel ID   : 101  
Timestamp   : 10:20  
Parcel #101 added to pickup queue.
```

Press Enter to continue...

```
[ Dispatch ]
```

- ```
-----  
1. Add Pickup Request  
2. View Pickup Queue  
3. Move to Dispatch Queue  
4. View Dispatch Queue  
5. Sort Dispatch by Weight (Heaviest First)  
6. Deliver Next Parcel (Simulate)  
0. Back  
-----
```

```
Choice: 1  
-----
```

```
Add Pickup Request  
-----
```

```
Parcel ID   : 102  
Timestamp   : 10:21  
Parcel #102 added to pickup queue.
```

View Pick Up Queue:

```
[ Dispatch ]
-----
1. Add Pickup Request
2. View Pickup Queue
3. Move to Dispatch Queue
4. View Dispatch Queue
5. Sort Dispatch by Weight (Heaviest First)
6. Deliver Next Parcel (Simulate)
0. Back
-----
Choice: 2
-----
Pickup Queue
-----
[1] Parcel #101
[2] Parcel #102
[3] Parcel #103
Press Enter to continue...|
```

Moving to Dispatch Queue:

```
[ Dispatch ]
-----
1. Add Pickup Request
2. View Pickup Queue
3. Move to Dispatch Queue
4. View Dispatch Queue
5. Sort Dispatch by Weight (Heaviest First)
6. Deliver Next Parcel (Simulate)
0. Back
-----
Choice: 3
-----
Move Pickup -> Dispatch
-----
Timestamp : 10:43
Parcel #101 moved to dispatch queue.
Press Enter to continue...
```

```
-----
Choice: 3
-----
Move Pickup -> Dispatch
-----
Timestamp : 10:45
Parcel #102 moved to dispatch queue.
```

View Dispatch Queue:

```
[ Dispatch ]
-----
1. Add Pickup Request
2. View Pickup Queue
3. Move to Dispatch Queue
4. View Dispatch Queue
5. Sort Dispatch by Weight (Heaviest First)
6. Deliver Next Parcel (Simulate)
0. Back
-----
Choice: 4
-----
Dispatch Queue
-----
[1] Parcel #101 | 2.50 kg | Colombo -> Gampaha
[2] Parcel #102 | 1.00 kg | Gampaha -> Galle
[3] Parcel #103 | 4.00 kg | Galle -> Colombo
Press Enter to continue...|
```

Sort Dispatch Queue:

```
[ Dispatch ]
-----
1. Add Pickup Request
2. View Pickup Queue
3. Move to Dispatch Queue
4. View Dispatch Queue
5. Sort Dispatch by Weight (Heaviest First)
6. Deliver Next Parcel (Simulate)
0. Back
-----
Choice: 5
-----
Sort Dispatch Queue by Weight (Heaviest First)
-----
Dispatch queue sorted by weight (heaviest first).

[1] Parcel #103 | 4.00 kg | Galle -> Colombo
[2] Parcel #101 | 2.50 kg | Colombo -> Gampaha
[3] Parcel #102 | 1.00 kg | Gampaha -> Galle
Press Enter to continue...|
```

## Auto Updating Tracking Entries:

```
[ Tracking ]
-----
1. Add Tracking Update
2. View Tracking History
0. Back
-----
Choice: 2
-----
Tracking History
-----
Parcel ID: 101

History for Parcel #101 (Colombo -> Gampaha):

[1] Status: Registered   Location: Colombo   Time: 09:00
[2] Status: Packaging     Location: Colombo   Time: 10:05
[3] Status: Pickup Requested Location: Colombo   Time: 10:20
[4] Status: Out for Delivery Location: Colombo   Time: 10:43

Press Enter to continue...|
```

## Simulate Delivery:

```
Choice: 4

[ Dispatch ]
-----
1. Add Pickup Request
2. View Pickup Queue
3. Move to Dispatch Queue
4. View Dispatch Queue
5. Sort Dispatch by Weight (Heaviest First)
6. Deliver Next Parcel (Simulate)
0. Back
-----
Choice: 6
-----
Simulate Delivery
-----

Delivering Parcel #103  (Galle -> Colombo)

Route taken:

Step 1 : Galle
Step 2 : Colombo

Delivery Timestamp : 12:04
Parcel #103 delivered and moved to completed records.

Press Enter to continue...|
```

Removed delivered Parcel references from active lists:

## [ Parcels ]

- 1. Register Parcel
- 2. Delete Parcel
- 3. Display All Parcels
- 0. Back

Choice: 3

## Parcel Records

ID: 101   Sender: Chathushi   Receiver: Sadeesha  
Weight: 2.50 kg   Colombo   -->   Gampaha

ID: 102   Sender: Manula   Receiver: Supun  
Weight: 1.00 kg   Gampaha   -->   Galle

ID: 104   Sender: Thisara   Receiver: Dahamsa  
Weight: 6.00 kg   Anuradhapura   -->   Galle

ID: 106   Sender: Sunil   Receiver: Nimal  
Weight: 2.50 kg   Anuradhapura   -->   Rathnapura

Press Enter to continue...|

## [ Tracking ]

- 1. Add Tracking Update
- 2. View Tracking History
- 0. Back

Choice: 2

## Tracking History

Parcel ID: 103  
Parcel #103 not found.

Press Enter to continue...|

## Completed Deliveries Record with full tracking records:

```
MAIN MENU
-----
1. Delivery Centers
2. Parcels
3. Tracking
4. Dispatch & Delivery
5. Completed Deliveries
6. Load Demo Data
0. Exit
-----
Choice: 5

[ Completed Deliveries ]
-----
1. View Completed Deliveries
2. Clear Completed Deliveries
0. Back
-----
Choice: 1

Completed Deliveries
-----
[1] Parcel #103 | Galle -> Colombo | 4.00 kg | Dahamsa -> Bindusara
    [1] Registered | Galle | 09:00
    [2] Pickup Requested | Galle | 10:27
    [3] Out for Delivery | Galle | 10:49
    [4] Delivered | Colombo | 12:04
-----
Press Enter to continue...|
```

After Delivering the rest in dispatch queue (Most recent on top):

```
[ Completed Deliveries ]
-----
1. View Completed Deliveries
2. Clear Completed Deliveries
0. Back
-----
Choice: 1

Completed Deliveries
-----
[1] Parcel #102 | Gampaha -> Galle | 1.00 kg | Manula -> Supun
    [1] Registered | Gampaha | 09:00
    [2] Pickup Requested | Gampaha | 10:21
    [3] Out for Delivery | Gampaha | 10:45
    [4] Delivered | Galle | 13:16
-----
[2] Parcel #101 | Colombo -> Gampaha | 2.50 kg | Chathushi -> Sadeesha
    [1] Registered | Colombo | 09:00
    [2] Packaging | Colombo | 10:05
    [3] Pickup Requested | Colombo | 10:20
    [4] Out for Delivery | Colombo | 10:43
    [5] Delivered | Gampaha | 13:12
-----
[3] Parcel #103 | Galle -> Colombo | 4.00 kg | Dahamsa -> Bindusara
    [1] Registered | Galle | 09:00
    [2] Pickup Requested | Galle | 10:27
    [3] Out for Delivery | Galle | 10:49
    [4] Delivered | Colombo | 12:04
-----
Press Enter to continue...|
```

Clear Completed Deliveries Stack:

```
[ Completed Deliveries ]
```

- ```
-----  
1. View Completed Deliveries  
2. Clear Completed Deliveries  
0. Back  
-----
```

```
Choice: 2  
-----
```

```
Clear Completed Deliveries  
-----
```

```
Completed deliveries cleared.
```

```
Press Enter to continue...
```

```
[ Completed Deliveries ]
```

- ```
-----  
1. View Completed Deliveries  
2. Clear Completed Deliveries  
0. Back  
-----
```

```
Choice: 1  
-----
```

```
Completed Deliveries  
-----
```

```
No completed deliveries.
```

```
Press Enter to continue...|
```

## 05. Discussion

### Parcels : Doubly Linked List

Parcels are added and removed dynamically so a fixed size array would waste memory or run out of space. Doubly linked allows deletion with lesser time complexity compared to singly linked list since each node knows its previous node, therefore no traversal needed to find the previous node.

### Distribution Centers : Circular Doubly Linked List

Delivery centers form a physical ring route with no end point, circular structure directly models this real world layout. Doubly linked list enables traversing both forward and backward to compare two possible routes between any two centers and pick the shorter one. Inserting a center between two existing ones only requires reassigning four pointers regardless of list size.

### Tracking History : Singly Linked List per Parcel

Each parcel has its own independent tracking history that grows based on activities related to the parcel. Dynamic size makes linked list more appropriate than a fixed array since the number of centers it will travel to before reaching the customer varies.

Tracking is always appended at the end and read front to back order. Therefore, no backward traversal would be needed. So singly linked is sufficient and simpler than doubly linked lists for the required purpose.

### Pickup : Queue

Queue directly models the real-world principle of first come first served (FIFO) , first customer to request pickup gets served first. Enqueuing at rear and dequeuing from front follows this order naturally by the nature of a queue. This is not sorted by any criteria to protect service fairness.

### Dispatch : Queue

Same queue structure as pickup. Once parcels are collected, delivery order can be optimized without affecting the fairness of pickup order.

### Insertion Sort on Dispatch Queue

Bubble sorts require a higher number of swapping nodes which are expensive and selection sort would ignore the best case and still go till the unsorted portion is cleared. Therefore, the best method is insertion sorting.

Insertion sort is an in place sorting algorithm, the queue is sorted within itself without building a secondary list or using extra memory proportional to input size, rather a fixed number of temporary variables till the sorting is completed.

Best case time complexity is  $O(n)$  when the queue is already nearly sorted and the worst case is  $O(n^2)$  which is acceptable given that a dispatch queue in a real depot holds a small number of parcels at a given time.

### Route : *Stack*

Delivery route is built step by step and then used step by step in the same order. When the parcel reaches a center, we can easily pop the route stack head to have the next center it should be transported to as the top of the stack again and if by any mistake it was transported to the wrong center we can push the relevant center to the stack as to where the parcel is at the moment. This would not have been possible with a queue as what we add is added to the rear and not to the front which we could've used as the current center.

### Completed Deliveries : *Stack*

Most recent delivery is most likely to be reviewed, stack gives  $O(1)$  access to it at the top. Storing the original parcel pointer with tracking history intact avoids duplicating data.

## 06. Conclusion

### Lessons Learned:

- Pointer based data structures require careful management of linking and unlinking nodes in both directions without losing references to surrounding nodes.
- Circular doubly linked list naturally has a ring route and enables bidirectional distance comparison without special end points.
- Embedding one data structure inside another (Tracking list inside each parcel node) keeps related data co-located and removes the need for a separate lookup.
- A stack's LIFO behavior can be used for a predefined order which might face unexpected real time changes unlike a queue which can enqueue the change but has to wait till the all the other predefined elements are dequeued.
- Separating utility functions like printLine, readInt, readFloat and readLine into their own functions reduces repeated code, makes each function focused on a single responsibility, and makes input handling consistent across the whole program
- A char array cannot be assigned with = because it is a fixed block of memory, not a variable, so strcpy must be used to copy characters one by one into the array.

### Challenges and Solutions :

- scanf leaving newline characters in the buffer caused inputs to be skipped or read incorrectly. Solved by writing a dedicated flushInput function called after every scanf
- Multi-word strings like "On the Way" were being split into separate fields by scanf since it stops at whitespace. Solved by switching to fgets via readLine which reads the entire line including spaces.
- The first center added to the circular list incorrectly asked for a wrap-around distance back to itself which had no meaning. Solved by detecting when only one center exists and skipping that prompt.
- The initial delivery simulation always took the forward route regardless of distance. Solved by calling calcRoutes to compare both directions and selecting the shorter path.
- The route stack originally used an extra temporary stack, pushing elements twice, which made it unnecessarily complex. It was simplified by directly traversing from destination to source and pushing onto the route stack, achieving the correct order more efficiently.

## 07. Source Code

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>


struct TrackingNode {

    char status[100];

    char location[100];

    char timestamp[50];

    struct TrackingNode* next;

};


struct Parcel {

    int id;

    char sender[50];

    char receiver[50];

    float weight;

    char source[50];

    char destination[50];

    struct TrackingNode* trackingHead;

    struct Parcel* next;

    struct Parcel* prev;

};


struct CenterNode {

    char name[50];

    float distToNext;

    float distToPrev;

    struct CenterNode* next;

    struct CenterNode* prev;
```

```
};
```

```
struct QueueNode {  
    int parcelID;  
    struct QueueNode* next;  
};
```

```
struct StackNode {  
    char center[50];  
    struct StackNode* next;  
};
```

```
struct CompletedNode {  
    struct Parcel* parcel;  
    struct CompletedNode* next;  
};
```

```
struct ParcelList {  
    struct Parcel* head;  
};
```

```
struct CenterList {  
    struct CenterNode* head;  
    struct CenterNode* tail;  
    int count;  
};
```

```
struct Queue {  
    struct QueueNode* front;  
    struct QueueNode* rear;  
};
```

```
struct Stack {  
    struct StackNode* top;  
};
```

```
struct CompletedStack {  
    struct CompletedNode* top;  
};
```

```
struct ParcelList parcels = {NULL};  
struct CenterList centers = {NULL, NULL, 0};  
struct Queue pickupQueue = {NULL, NULL};  
struct Queue dispatchQueue = {NULL, NULL};  
struct Stack routeStack = {NULL};  
struct CompletedStack completed = {NULL};  
int demoLoaded = 0;
```

```
void printLine() {  
    printf("-----\n");  
}
```

```
void flushInput() {  
    int c;  
    while ((c = getchar()) != '\n' && c != EOF);  
}
```

```
void pressEnter() {  
    printf("\nPress Enter to continue...");  
    fflush(stdout);  
    flushInput();  
}
```

```

void readLine(char* buf, int size) {
    if (fgets(buf, size, stdin) != NULL) {
        int len = strlen(buf);
        if (len > 0 && buf[len - 1] == '\n')
            buf[len - 1] = '\0';
    }
}

```

```

int readInt(int* out) {
    if (scanf("%d", out) != 1) {
        flushInput();
        return 0;
    }
    flushInput();
    return 1;
}

```

```

int readFloat(float* out) {
    if (scanf("%f", out) != 1) {
        flushInput();
        return 0;
    }
    flushInput();
    return 1;
}

```

```

void appendTracking(struct Parcel* p, const char* status, const char* location, const char*
timestamp) {
    struct TrackingNode* node = malloc(sizeof(struct TrackingNode));
    strcpy(node->status, status);

```

```
strcpy(node->location, location);
strcpy(node->timestamp, timestamp);
node->next = NULL;
```

```
if (p->trackingHead == NULL) {
    p->trackingHead = node;
} else {
    struct TrackingNode* t = p->trackingHead;
    while (t->next != NULL)
        t = t->next;
    t->next = node;
}
}
```

```
struct Parcel* findParcel(int id) {
    struct Parcel* temp = parcels.head;
    while (temp != NULL) {
        if (temp->id == id)
            return temp;
        temp = temp->next;
    }
    return NULL;
}
```

```
void freeTracking(struct TrackingNode* head) {
    while (head != NULL) {
        struct TrackingNode* t = head;
        head = head->next;
        free(t);
    }
}
```

```

void detachParcelFromList(int id) {
    struct Parcel* p = findParcel(id);
    if (p == NULL)
        return;

    if (p->prev != NULL)
        p->prev->next = p->next;
    else
        parcels.head = p->next;

    if (p->next != NULL)
        p->next->prev = p->prev;

    p->next = NULL;
    p->prev = NULL;
}

```

```

void removeFromQueue(struct Queue* q, int id) {
    struct QueueNode* curr = q->front;
    struct QueueNode* prev = NULL;
    while (curr != NULL) {
        if (curr->parcelID == id) {
            if (prev != NULL)
                prev->next = curr->next;
            else
                q->front = curr->next;
            if (curr == q->rear)
                q->rear = prev;
            free(curr);
            return;
        }
        prev = curr;
        curr = curr->next;
    }
}

```

```

    }

    prev = curr;
    curr = curr->next;
}
}

```

```

void pushCompleted(struct Parcel* p) {
    struct CompletedNode* node = malloc(sizeof(struct CompletedNode));
    node->parcel = p;
    node->next = completed.top;
    completed.top = node;
}

```

```

void addParcel() {
    struct Parcel* p = malloc(sizeof(struct Parcel));

    printLine();
    printf(" Register New Parcel\n");
    printLine();

    printf(" Parcel ID  : ");
    if (!readInt(&p->id)) {
        printf(" Invalid input.\n");
        free(p);
        pressEnter();
        return;
    }
}

```

```

if (findParcel(p->id) != NULL) {
    printf(" A parcel with ID %d already exists.\n", p->id);
    free(p);
}

```

```

    pressEnter();
    return;
}

printf(" Sender    : "); readLine(p->sender, sizeof(p->sender));
printf(" Receiver  : "); readLine(p->receiver, sizeof(p->receiver));
printf(" Weight (kg) : "); readFloat(&p->weight);
printf(" Source    : "); readLine(p->source, sizeof(p->source));
printf(" Destination : "); readLine(p->destination, sizeof(p->destination));

char timestamp[50];
printf(" Timestamp  : "); readLine(timestamp, sizeof(timestamp));

p->trackingHead = NULL;
p->next = NULL;
p->prev = NULL;

appendTracking(p, "Registered", p->source, timestamp);

if (parcels.head == NULL) {
    parcels.head = p;
} else {
    struct Parcel* temp = parcels.head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = p;
    p->prev = temp;
}

printf("\n Parcel #%d registered successfully.\n", p->id);
pressEnter();

```

```
}
```

```
void deleteParcel() {
```

```
    int id;
```

```
    printLine();
```

```
    printf(" Delete Parcel\n");
```

```
    printLine();
```

```
    printf(" Parcel ID: ");
```

```
    if (!readInt(&id)) {
```

```
        printf(" Invalid input.\n");
```

```
        pressEnter();
```

```
        return;
```

```
    }
```

```
    struct Parcel* p = findParcel(id);
```

```
    if (p == NULL) {
```

```
        printf(" Parcel #%%d not found.\n", id);
```

```
        pressEnter();
```

```
        return;
```

```
    }
```

```
    removeFromQueue(&pickupQueue, id);
```

```
    removeFromQueue(&dispatchQueue, id);
```

```
    detachParcelFromList(id);
```

```
    freeTracking(p->trackingHead);
```

```
    free(p);
```

```
    printf(" Parcel #%%d deleted and removed from all queues.\n", id);
```

```
    pressEnter();
```

```
}
```

```

void displayParcels() {
    printLine();
    printf(" Parcel Records\n");
    printLine();

    if (parcels.head == NULL) {
        printf(" No parcels registered.\n");
        pressEnter();
        return;
    }

    struct Parcel* p = parcels.head;
    while (p != NULL) {
        printf(" ID: %d Sender: %s Receiver: %s\n", p->id, p->sender, p->receiver);
        printf(" Weight: %.2f kg   %s --> %s\n", p->weight, p->source, p->destination);
        printLine();
        p = p->next;
    }
    pressEnter();
}

```

```

void addTracking() {
    int id;
    printLine();
    printf(" Add Tracking Update\n");
    printLine();
    printf(" Parcel ID : ");
    if (!readInt(&id)) {
        printf(" Invalid input.\n");
        pressEnter();
        return;
    }
}

```

```
}
```

```
struct Parcel* p = findParcel(id);
```

```
if (p == NULL) {
```

```
    printf(" Parcel #%%d not found.\n", id);
```

```
    pressEnter();
```

```
    return;
```

```
}
```

```
char status[100], location[100], timestamp[50];
```

```
printf(" Status   : "); readLine(status, sizeof(status));
```

```
printf(" Location  : "); readLine(location, sizeof(location));
```

```
printf(" Timestamp : "); readLine(timestamp, sizeof(timestamp));
```

```
appendTracking(p, status, location, timestamp);
```

```
printf(" Tracking update added for parcel #%%d.\n", id);
```

```
pressEnter();
```

```
}
```

```
void viewTracking() {
```

```
    int id;
```

```
    printLine();
```

```
    printf(" Tracking History\n");
```

```
    printLine();
```

```
    printf(" Parcel ID: ");
```

```
    if (!readInt(&id)) {
```

```
        printf(" Invalid input.\n");
```

```
        pressEnter();
```

```
        return;
```

```
}
```

```

struct Parcel* p = findParcel(id);
if (p == NULL) {
    printf(" Parcel #%%d not found.\n", id);
    pressEnter();
    return;
}

```

```

if (p->trackingHead == NULL) {
    printf(" No tracking history for parcel #%%d.\n", id);
    pressEnter();
    return;
}

```

```

printf("\n History for Parcel #%%d (%%s -> %%s):\n\n", id, p->source, p->destination);
struct TrackingNode* t = p->trackingHead;
int step = 1;
while (t != NULL) {
    printf(" [%%d] Status: %%s Location: %%s Time: %%s\n",
           step++, t->status, t->location, t->timestamp);
    t = t->next;
}
pressEnter();
}

```

```

struct CenterNode* findCenter(const char* name) {
    if (centers.head == NULL)
        return NULL;
    struct CenterNode* curr = centers.head;
    do {
        if (strcmp(curr->name, name) == 0)

```

```

        return curr;

    curr = curr->next;
} while (curr != centers.head);
return NULL;
}

```

```

void insertCenterAtEnd(const char* name, float distFromPrev, float distToHead) {
    struct CenterNode* node = malloc(sizeof(struct CenterNode));
    strcpy(node->name, name);
    node->distToNext = 0;
    node->distToPrev = 0;
    node->next = NULL;
    node->prev = NULL;

    if (centers.head == NULL) {
        centers.head = node;
        node->next = node;
        node->prev = node;
    } else {
        struct CenterNode* tail = centers.head->prev;
        tail->next = node;
        node->prev = tail;
        node->next = centers.head;
        centers.head->prev = node;

        node->distToPrev = distFromPrev;
        tail->distToNext = distFromPrev;
        node->distToNext = distToHead;
        centers.head->distToPrev = distToHead;
    }
    centers.tail = node;
}

```

```
    centers.count++;  
}
```

```
void addCenter() {  
    printLine();  
    printf(" Add Delivery Center\n");  
    printLine();  
  
    char name[50];  
    printf(" Center Name : "); readLine(name, sizeof(name));  
  
    if (findCenter(name) != NULL) {  
        printf(" A center named '%s' already exists.\n", name);  
        pressEnter();  
        return;  
    }  
}
```

```
struct CenterNode* node = malloc(sizeof(struct CenterNode));  
strcpy(node->name, name);  
node->distToNext = 0;  
node->distToPrev = 0;  
node->next = NULL;  
node->prev = NULL;
```

```
if (centers.head == NULL) {  
    centers.head = node;  
    node->next = node;  
    node->prev = node;  
    centers.tail = node;  
    centers.count++;  
    printf(" Center '%s' added.\n", name);  
}
```

```
    pressEnter();  
    return;  
}
```

```
char afterName[50];  
printf(" Located after which center (default: %s, press Enter to use default): ",  
       centers.tail->name);  
readLine(afterName, sizeof(afterName));
```

```
struct CenterNode* afterNode = NULL;  
if (strlen(afterName) == 0) {  
    afterNode = centers.tail;  
} else {  
    afterNode = findCenter(afterName);  
    if (afterNode == NULL) {  
        printf(" Center '%s' not found. Aborting.\n", afterName);  
        free(node);  
        pressEnter();  
        return;  
    }  
}
```

```
struct CenterNode* beforeNode = afterNode->next;
```

```
float distAfterToNew, distNewToBefore;  
printf(" Distance from %s to %s (km): ", afterNode->name, name);  
readFloat(&distAfterToNew);  
printf(" Distance from %s to %s (km): ", name, beforeNode->name);  
readFloat(&distNewToBefore);
```

```
afterNode->next = node;
```

```
node->prev = afterNode;
node->next = beforeNode;
beforeNode->prev = node;
```

```
afterNode->distToNext = distAfterToNew;
node->distToPrev = distAfterToNew;
node->distToNext = distNewToBefore;
beforeNode->distToPrev = distNewToBefore;
```

```
if (afterNode == centers.tail)
    centers.tail = node;
```

```
centers.count++;
printf(" Center '%s' inserted after '%s'.\n", name, afterNode->name);
pressEnter();
```

```
}
```

```
void displayCenters() {
    printLine();
    printf(" Delivery Centers (Circular)\n");
    printLine();
```

```
if (centers.head == NULL) {
    printf(" No centers registered.\n");
    pressEnter();
    return;
}
```

```
struct CenterNode* curr = centers.head;
int i = 1;
do {
```

```

printf(" [%d] %s --> %s (%.1f km)\n",
        i++, curr->name, curr->next->name, curr->distToNext);
curr = curr->next;
} while (curr != centers.head);

pressEnter();
}

```

```

int calcRoutes(const char* src, const char* dst,
               char* fwdPath, float* fwdDist,
               char* bwdPath, float* bwdDist) {

    struct CenterNode* srcNode = findCenter(src);
    if (srcNode == NULL)
        return 0;
    if (findCenter(dst) == NULL)
        return 0;

    *fwdDist = 0;
    fwdPath[0] = '\0';
    struct CenterNode* curr = srcNode;
    do {
        strcat(fwdPath, curr->name);
        if (strcmp(curr->name, dst) == 0)
            break;
        strcat(fwdPath, " -> ");
        *fwdDist += curr->distToNext;
        curr = curr->next;
    } while (curr != srcNode);

    *bwdDist = 0;

```

```

bwdPath[0] = '\0';
curr = srcNode;
do {
    strcat(bwdPath, curr->name);
    if (strcmp(curr->name, dst) == 0)
        break;
    strcat(bwdPath, " -> ");
    *bwdDist += curr->distToPrev;
    curr = curr->prev;
} while (curr != srcNode);

return 1;
}

void findBestRoute() {
    char src[50], dst[50];
    printLine();
    printf(" Find Best Route Between Centers\n");
    printLine();

    if (centers.head == NULL || centers.count < 2) {
        printf(" Need at least 2 centers.\n");
        pressEnter();
        return;
    }

    printf(" Source Center    : "); readLine(src, sizeof(src));
    printf(" Destination Center : "); readLine(dst, sizeof(dst));

    char fwdPath[500], bwdPath[500];
    float fwdDist, bwdDist;

```

```
if (calcRoutes(src, dst, fwdPath, &fwdDist, bwdPath, &bwdDist) == 0) {  
    printf(" One or both centers not found.\n");  
    pressEnter();  
    return;  
}
```

```
printf("\n Forward : %s (%.2f km)\n", fwdPath, fwdDist);  
printf(" Backward : %s (%.2f km)\n", bwdPath, bwdDist);
```

```
if (fwdDist <= bwdDist)  
    printf("\n Best route: %s (%.2f km)\n", fwdPath, fwdDist);  
else  
    printf("\n Best route: %s (%.2f km)\n", bwdPath, bwdDist);  
  
pressEnter();  
}
```

```
void enqueuePickup() {  
    int id;  
    printLine();  
    printf(" Add Pickup Request\n");  
    printLine();  
    printf(" Parcel ID : ");  
    if (!readInt(&id)) {  
        printf(" Invalid input.\n");  
        pressEnter();  
        return;  
    }  
}
```

```
struct Parcel* p = findParcel(id);
```

```
if (p == NULL) {  
    printf(" Parcel #%%d not found. Register it first.\n", id);  
    pressEnter();  
    return;  
}
```

```
char timestamp[50];  
printf(" Timestamp : "); readLine(timestamp, sizeof(timestamp));
```

```
struct QueueNode* node = malloc(sizeof(struct QueueNode));  
node->parcelID = id;  
node->next = NULL;
```

```
if (pickupQueue.rear == NULL) {  
    pickupQueue.front = pickupQueue.rear = node;  
} else {  
    pickupQueue.rear->next = node;  
    pickupQueue.rear = node;  
}
```

```
appendTracking(p, "Pickup Requested", p->source, timestamp);
```

```
printf(" Parcel #%%d added to pickup queue.\n", id);  
pressEnter();  
}
```

```
void displayPickupQueue() {  
    printLine();  
    printf(" Pickup Queue\n");  
    printLine();  
}
```

```

if (pickupQueue.front == NULL) {
    printf(" Pickup queue is empty.\n");
    pressEnter();
    return;
}

```

```

struct QueueNode* q = pickupQueue.front;
int pos = 1;
while (q != NULL) {
    printf(" [%d] Parcel #%d\n", pos++, q->parcelID);
    q = q->next;
}
pressEnter();
}

```

```

void moveToDispatch() {
    printLine();
    printf(" Move Pickup -> Dispatch\n");
    printLine();
}

```

```

if (pickupQueue.front == NULL) {
    printf(" Pickup queue is empty.\n");
    pressEnter();
    return;
}

```

```

char timestamp[50];
printf(" Timestamp : "); readLine(timestamp, sizeof(timestamp));

```

```

int id = pickupQueue.front->parcelID;
struct QueueNode* temp = pickupQueue.front;

```

```
pickupQueue.front = pickupQueue.front->next;
```

```
if (pickupQueue.front == NULL)
```

```
    pickupQueue.rear = NULL;
```

```
free(temp);
```

```
struct QueueNode* node = malloc(sizeof(struct QueueNode));
```

```
node->parcelID = id;
```

```
node->next = NULL;
```

```
if (dispatchQueue.rear == NULL) {
```

```
    dispatchQueue.front = dispatchQueue.rear = node;
```

```
} else {
```

```
    dispatchQueue.rear->next = node;
```

```
    dispatchQueue.rear = node;
```

```
}
```

```
struct Parcel* p = findParcel(id);
```

```
if (p != NULL)
```

```
    appendTracking(p, "Out for Delivery", p->source, timestamp);
```

```
printf(" Parcel #%d moved to dispatch queue.\n", id);
```

```
pressEnter();
```

```
}
```

```
void displayDispatchQueue() {
```

```
    printLine();
```

```
    printf(" Dispatch Queue\n");
```

```
    printLine();
```

```
if (dispatchQueue.front == NULL) {
```

```
    printf(" Dispatch queue is empty.\n");
```

```
    pressEnter();  
    return;  
}
```

```
struct QueueNode* q = dispatchQueue.front;  
int pos = 1;  
while (q != NULL) {  
    struct Parcel* p = findParcel(q->parcelID);  
    if (p != NULL)  
        printf(" [%d] Parcel #%d | %.2f kg | %s -> %s\n",  
            pos, q->parcelID, p->weight, p->source, p->destination);  
    else  
        printf(" [%d] Parcel #%d\n", pos, q->parcelID);  
    pos++;  
    q = q->next;  
}  
pressEnter();  
}
```

```
void sortDispatchByWeight() {  
    printLine();  
    printf(" Sort Dispatch Queue by Weight (Heaviest First)\n");  
    printLine();  
  
    if (dispatchQueue.front == NULL) {  
        printf(" Dispatch queue is empty.\n");  
        pressEnter();  
        return;  
    }  
  
    struct QueueNode* sortedTail = dispatchQueue.front;
```

```

while (sortedTail->next != NULL) {
    struct QueueNode* curr = sortedTail->next;
    struct Parcel* currParcel = findParcel(curr->parcelID);
    float currWeight = (currParcel != NULL) ? currParcel->weight : 0;

    struct Parcel* frontParcel = findParcel(dispatchQueue.front->parcelID);
    float frontWeight = (frontParcel != NULL) ? frontParcel->weight : 0;

    if (currWeight >= frontWeight) {
        sortedTail->next = curr->next;
        curr->next = dispatchQueue.front;
        dispatchQueue.front = curr;
    } else {
        struct QueueNode* s = dispatchQueue.front;
        while (s->next != curr) {
            struct Parcel* nextParcel = findParcel(s->next->parcelID);
            float nextWeight = (nextParcel != NULL) ? nextParcel->weight : 0;
            if (currWeight >= nextWeight)
                break;
            s = s->next;
        }
        if (s->next != curr) {
            sortedTail->next = curr->next;
            curr->next = s->next;
            s->next = curr;
        } else {
            sortedTail = sortedTail->next;
        }
    }
}
}

```

```
dispatchQueue.rear = sortedTail;
```

```
printf(" Dispatch queue sorted by weight (heaviest first).\n\n");
```

```
struct QueueNode* q = dispatchQueue.front;
```

```
int pos = 1;
```

```
while (q != NULL) {
```

```
    struct Parcel* p = findParcel(q->parcelID);
```

```
    if (p != NULL)
```

```
        printf(" [%d] Parcel #%d | %.2f kg | %s -> %s\n",
```

```
            pos, q->parcelID, p->weight, p->source, p->destination);
```

```
    pos++;
```

```
    q = q->next;
```

```
}
```

```
pressEnter();
```

```
}
```

```
void pushRoute(const char* center) {
```

```
    struct StackNode* node = malloc(sizeof(struct StackNode));
```

```
    strcpy(node->center, center);
```

```
    node->next = routeStack.top;
```

```
    routeStack.top = node;
```

```
}
```

```
void simulateDelivery() {
```

```
    printLine();
```

```
    printf(" Simulate Delivery\n");
```

```
    printLine();
```

```
    if (dispatchQueue.front == NULL) {
```

```

printf(" Dispatch queue is empty. No parcels to deliver.\n");
pressEnter();
return;
}

int id = dispatchQueue.front->parcelID;
struct QueueNode* temp = dispatchQueue.front;
dispatchQueue.front = dispatchQueue.front->next;
if (dispatchQueue.front == NULL)
    dispatchQueue.rear = NULL;
free(temp);

struct Parcel* p = findParcel(id);
if (p == NULL) {
    printf(" Parcel #%d record missing. Skipping.\n", id);
    pressEnter();
    return;
}

printf("\n Delivering Parcel #%d (%s -> %s)\n\n", id, p->source, p->destination);

if (centers.head == NULL) {
    pushRoute(p->destination);
    pushRoute(p->source);
} else {
    char fwdPath[500], bwdPath[500];
    float fwdDist, bwdDist;
    int found = calcRoutes(p->source, p->destination,
                          fwdPath, &fwdDist, bwdPath, &bwdDist);

    if (found == 0) {

```

```

    pushRoute(p->destination);
    pushRoute(p->source);
} else {
    int useForward = (fwdDist <= bwdDist);
    struct CenterNode* dstNode = findCenter(p->destination);
    struct CenterNode* c = dstNode;
    int steps = 0;

    do {
        pushRoute(c->name);
        if (strcmp(c->name, p->source) == 0)
            break;
        c = useForward ? c->prev : c->next;
        steps++;
        if (steps > centers.count + 1)
            break;
    } while (c != dstNode);
}

printf(" Route taken:\n\n");
int step = 1;
while (routeStack.top != NULL) {
    printf(" Step %d : %s\n", step++, routeStack.top->center);
    struct StackNode* top = routeStack.top;
    routeStack.top = routeStack.top->next;
    free(top);
}

char timestamp[50];
printf("\n Delivery Timestamp : "); readLine(timestamp, sizeof(timestamp));

```

```
appendTracking(p, "Delivered", p->destination, timestamp);
```

```
detachParcelFromList(id);
```

```
removeFromQueue(&pickupQueue, id);
```

```
pushCompleted(p);
```

```
printf(" Parcel #%d delivered and moved to completed records.\n", id);
```

```
pressEnter();
```

```
}
```

```
void viewCompleted() {
```

```
    printLine();
```

```
    printf(" Completed Deliveries\n");
```

```
    printLine();
```

```
    if (completed.top == NULL) {
```

```
        printf(" No completed deliveries.\n");
```

```
        pressEnter();
```

```
        return;
```

```
    }
```

```
    struct CompletedNode* cn = completed.top;
```

```
    int i = 1;
```

```
    while (cn != NULL) {
```

```
        struct Parcel* p = cn->parcel;
```

```
        printf(" [%d] Parcel #%d | %s -> %s | %.2f kg | %s -> %s\n",
```

```
            i++, p->id, p->source, p->destination, p->weight, p->sender, p->receiver);
```

```
        struct TrackingNode* t = p->trackingHead;
```

```
        int step = 1;
```

```

while (t != NULL) {
    printf("    [%d] %s | %s | %s\n", step++, t->status, t->location, t->timestamp);
    t = t->next;
}
printLine();
cn = cn->next;
}
pressEnter();
}

```

```

void clearCompleted() {
    printLine();
    printf(" Clear Completed Deliveries\n");
    printLine();
}

```

```

if (completed.top == NULL) {
    printf(" Nothing to clear.\n");
    pressEnter();
    return;
}

```

```

while (completed.top != NULL) {
    struct CompletedNode* cn = completed.top;
    completed.top = completed.top->next;
    freeTracking(cn->parcel->trackingHead);
    free(cn->parcel);
    free(cn);
}

```

```

printf(" Completed deliveries cleared.\n");
pressEnter();

```

```
}
```

```
void loadDemoData() {
```

```
    printLine();
```

```
    printf(" Load Demo Data\n");
```

```
    printLine();
```

```
    if (demoLoaded) {
```

```
        printf(" Demo data is already loaded.\n");
```

```
        pressEnter();
```

```
        return;
```

```
    }
```

```
    char timestamp[50];
```

```
    printf(" Timestamp for all demo entries : "); readLine(timestamp, sizeof(timestamp));
```

```
    insertCenterAtEnd("Colombo", 0, 120.0);
```

```
    insertCenterAtEnd("Gampaha", 35.0, 505.0);
```

```
    insertCenterAtEnd("Anuradhapura", 180.0, 290.0);
```

```
    insertCenterAtEnd("Galle", 290.0, 120.0);
```

```
    printf(" Centers: Colombo, Gampaha, Anuradhapura, Galle\n");
```

```
    struct Parcel* p1 = malloc(sizeof(struct Parcel));
```

```
    p1->id = 101;
```

```
    strcpy(p1->sender, "Chathushi");
```

```
    strcpy(p1->receiver, "Sadeesha");
```

```
    p1->weight = 2.5;
```

```
    strcpy(p1->source, "Colombo");
```

```
    strcpy(p1->destination, "Gampaha");
```

```
    p1->trackingHead = NULL;
```

```
p1->next = NULL;
p1->prev = NULL;
appendTracking(p1, "Registered", "Colombo", timestamp);
parcels.head = p1;
```

```
struct Parcel* p2 = malloc(sizeof(struct Parcel));
p2->id = 102;
strcpy(p2->sender, "Manula");
strcpy(p2->receiver, "Supun");
p2->weight = 1.0;
strcpy(p2->source, "Gampaha");
strcpy(p2->destination, "Galle");
p2->trackingHead = NULL;
p2->next = NULL;
p2->prev = p1;
appendTracking(p2, "Registered", "Gampaha", timestamp);
p1->next = p2;
```

```
struct Parcel* p3 = malloc(sizeof(struct Parcel));
p3->id = 103;
strcpy(p3->sender, "Dahamsa");
strcpy(p3->receiver, "Bindusara");
p3->weight = 4.0;
strcpy(p3->source, "Galle");
strcpy(p3->destination, "Colombo");
p3->trackingHead = NULL;
p3->next = NULL;
p3->prev = p2;
appendTracking(p3, "Registered", "Galle", timestamp);
p2->next = p3;
```

```
struct Parcel* p4 = malloc(sizeof(struct Parcel));
p4->id = 104;
strcpy(p4->sender, "Thisara");
strcpy(p4->receiver, "Dahamsa");
p4->weight = 6.0;
strcpy(p4->source, "Anuradhapura");
strcpy(p4->destination, "Galle");
p4->trackingHead = NULL;
p4->next = NULL;
p4->prev = p3;
appendTracking(p4, "Registered", "Anuradhapura", timestamp);
p3->next = p4;
```

```
struct Parcel* p5 = malloc(sizeof(struct Parcel));
p5->id = 105;
strcpy(p5->sender, "Amal");
strcpy(p5->receiver, "Kamal");
p5->weight = 3.2;
strcpy(p5->source, "Colombo");
strcpy(p5->destination, "Anuradhapura");
p5->trackingHead = NULL;
p5->next = NULL;
p5->prev = p4;
appendTracking(p5, "Registered", "Colombo", timestamp);
p4->next = p5;
```

```
demoLoaded = 1;
printf(" Parcels: #101 (Colombo->Gampaha, 2.5kg), #102 (Gampaha->Galle, 1.0kg),\n");
printf("      #103 (Galle->Colombo, 4.0kg), #104 (Anuradhapura->Galle, 6.0kg),\n");
printf("      #105 (Colombo->Anuradhapura, 3.2kg)\n");
printf("\n Demo data loaded.\n");
```

```
    pressEnter();  
}
```

```
void centersMenu() {  
    int choice;  
    do {  
        printf("\n [ Centers ]\n");  
        printLine();  
        printf(" 1. Add Delivery Center\n");  
        printf(" 2. Display Centers\n");  
        printf(" 3. Find Best Route\n");  
        printf(" 0. Back\n");  
        printLine();  
        printf(" Choice: "); readInt(&choice);  
  
        switch (choice) {  
            case 1: {  
                addCenter();  
                break;  
            }  
            case 2: {  
                displayCenters();  
                break;  
            }  
            case 3: {  
                findBestRoute();  
                break;  
            }  
            case 0: {  
                break;  
            }  
        }  
    }  
}
```

```
        default: {  
            printf(" Invalid choice.\n");  
        }  
    }  
} while (choice != 0);  
}
```

```
void parcelsMenu() {  
    int choice;  
    do {  
        printf("\n [ Parcels ]\n");  
        printLine();  
        printf(" 1. Register Parcel\n");  
        printf(" 2. Delete Parcel\n");  
        printf(" 3. Display All Parcels\n");  
        printf(" 0. Back\n");  
        printLine();  
        printf(" Choice: "); readInt(&choice);  
  
        switch (choice) {  
            case 1: {  
                addParcel();  
                break;  
            }  
            case 2: {  
                deleteParcel();  
                break;  
            }  
            case 3: {  
                displayParcels();  
                break;  
            }  
        }  
    } while (choice != 0);  
}
```

```
    }  
    case 0: {  
        break;  
    }  
    default: {  
        printf(" Invalid choice.\n");  
    }  
}  
} while (choice != 0);  
}
```

```
void trackingMenu() {  
    int choice;  
    do {  
        printf("\n [ Tracking ]\n");  
        printLine();  
        printf(" 1. Add Tracking Update\n");  
        printf(" 2. View Tracking History\n");  
        printf(" 0. Back\n");  
        printLine();  
        printf(" Choice: "); readInt(&choice);  
  
        switch (choice) {  
            case 1: {  
                addTracking();  
                break;  
            }  
            case 2: {  
                viewTracking();  
                break;  
            }  
        }  
    } while (choice != 0);  
}
```

```
        case 0: {
            break;
        }
        default: {
            printf(" Invalid choice.\n");
        }
    }
} while (choice != 0);
}
```

```
void dispatchMenu() {
    int choice;
    do {
        printf("\n [ Dispatch ]\n");
        printLine();
        printf(" 1. Add Pickup Request\n");
        printf(" 2. View Pickup Queue\n");
        printf(" 3. Move to Dispatch Queue\n");
        printf(" 4. View Dispatch Queue\n");
        printf(" 5. Sort Dispatch by Weight (Heaviest First)\n");
        printf(" 6. Deliver Next Parcel (Simulate)\n");
        printf(" 0. Back\n");
        printLine();
        printf(" Choice: "); readInt(&choice);

        switch (choice) {
            case 1: {
                enqueuePickup();
                break;
            }
            case 2: {
```

```

        displayPickupQueue();
        break;
    }
    case 3: {
        moveToDispatch();
        break;
    }
    case 4: {
        displayDispatchQueue();
        break;
    }
    case 5: {
        sortDispatchByWeight();
        break;
    }
    case 6: {
        simulateDelivery();
        break;
    }
    case 0: {
        break;
    }
    default: {
        printf(" Invalid choice.\n");
    }
}
} while (choice != 0);
}

```

```

void completedMenu() {
    int choice;

```

```

do {
    printf("\n [ Completed Deliveries ]\n");
    printLine();
    printf(" 1. View Completed Deliveries\n");
    printf(" 2. Clear Completed Deliveries\n");
    printf(" 0. Back\n");
    printLine();
    printf(" Choice: "); readInt(&choice);

    switch (choice) {
        case 1: {
            viewCompleted();
            break;
        }
        case 2: {
            clearCompleted();
            break;
        }
        case 0: {
            break;
        }
        default: {
            printf(" Invalid choice.\n");
        }
    }
} while (choice != 0);
}

```

```

int main() {
    int choice;

```

```
printf("\n=====\\n");
printf("    COURIER MANAGEMENT SYSTEM\\n");
printf("=====\\n");
```

```
do {
    printf("\\n MAIN MENU\\n");
    printLine();
    printf(" 1. Delivery Centers\\n");
    printf(" 2. Parcels\\n");
    printf(" 3. Tracking\\n");
    printf(" 4. Dispatch & Delivery\\n");
    printf(" 5. Completed Deliveries\\n");
    printf(" 6. Load Demo Data\\n");
    printf(" 0. Exit\\n");
    printLine();
    printf(" Choice: "); readInt(&choice);
```

```
switch (choice) {
    case 1: {
        centersMenu();
        break;
    }
    case 2: {
        parcelsMenu();
        break;
    }
    case 3: {
        trackingMenu();
        break;
    }
    case 4: {
```

```
        dispatchMenu();
        break;
    }
    case 5: {
        completedMenu();
        break;
    }
    case 6: {
        loadDemoData();
        break;
    }
    case 0: {
        printf("\n Goodbye.\n\n");
        break;
    }
    default: {
        printf(" Invalid choice.\n");
    }
}

} while (choice != 0);

return 0;
}
```